# Project NorConfig NAI V2.0

## Project Understanding

This is a **product line extension** of the existing NorConfig NAI V1.x production system to support roofing panels (NorSeam).

**Scope:**

- **System**: Extend existing NorConfig V1.x (production configurator for wall/insulated panels)
- **Integration Layer**: Operational and stable (Epicor, MES, planning systems) - minor schema updates may be required
- **Architecture**: Follow existing production patterns and stack
- **Timeline**: 4 months to August 2026 delivery
- **Technology**: Use existing production stack
- **Validation**: Develop validation capability for both architectural and roofing panels (strategy TBD)
- **UI/UX**: Extend existing interface rather than rebuild with enhanced workflow
    - New UI design excluded from initial scope due to tight timeline constraints
    - Leverages existing UI patterns and components for roofing panel inputs
    - Plan can be revised if timeline is extended or as phase 2 enhancement
    - Delivering functional capability with existing UI represents sound agility strategy: incremental value delivery over delayed perfection

**Key Implications:**

- **Feasibility**: Timeline is aggressive but achievable if existing architecture accommodates roofing panels without major refactoring
- **Risk Profile**: Extension complexity depends on current system design quality and backward compatibility requirements
- **Success Criteria**: Add roofing configuration capability without breaking existing wall panel functionality
- **Critical Factor**: Existing NorConfig V1.x architecture quality and extensibility determine project viability

---

## Source Document Analysis

### Clarification Points

- Norconfig NAI is a product configurator system for configurable architectural panels.
- The primary goal of the project is for adding the capability to configure roof panels.
- Business priorities clarified:
    - **Priority 1**: Enable roofing panel configuration capability
    - **Priority 2**: Prevent invalid panel configurations (both architectural and roofing panels)
- NorSeam is the name given to the roofing panel product line

**System Context:**

- NorConfig NAI V1.x is in production handling wall/insulated panel configurations
- Integration layer to Epicor, MES, planning systems is operational and stable

- This project extends the existing configurator to handle roofing panels (NorSeam)
- Integration changes minimal: add roofing-specific attributes to existing data flows

**Scope Characteristics:**

- Product line extension to existing production system
- Integration infrastructure already exists
- Must follow existing architectural patterns
- Uses production technology stack
- No net-new system build required

### Open Questions

- Will roofing panel be used for architectural projects on just exterior cold rooms?
- What are planned rollout dates?
- Who are stakeholders that can clarify the types of invalid configuration the configurator produces?
    - We need to have a good idea of errors that occurs for proposing best possible solution path.
- Who are stakeholders that can express flaws about the actual user interface?
    - We need to know good and bad points about current UI for proposing best possible solution path.
- Who are stakeholders that can clarify rollout dates?
- Who are stakeholders that can define needs for validation engine (actual and future extensibility in rules)

---

## Critical Analysis

### 🚨 Timeline Viability

**Status:** Project marked as "Idea to be Scoped" with **4 months to delivery** (August 2026)

**Critical Issues:**

- Phase 1 deadline (April 2026) has **already passed** as of April 6, 2026
- No evidence of scoping completion, technical analysis, or UML diagrams mentioned in milestones
- 16-week runway for extending existing system with:
    - Roofing panel configuration logic (new domain)
    - NorSeam-specific validation rules (new)
    - UI extensions for roofing-specific inputs (front/back, laps, notches, packaging)
    - Minor integration updates (add roofing attributes to existing flows)

**Assessment:** Timeline is **aggressive and at risk** due to:

- Validation capability must be developed (no existing validation for NAI architectural panels)
- Validation strategy undefined (complexity assessment needed to determine approach)
- Unknown: Current architecture extensibility for roofing panels
- Timeline does not account for foundational project governance (Brief, Analysis, Specification)

**Feasibility dependent on:**

- Validation requirements complexity (simple rules vs complex interdependencies)

- Architecture compatibility assessment results
- Establishing proper project workflow and governance structure

---

## 🎯 Scope Definition Gaps

**Missing Critical Elements:**

1. **No Roofing Panel Product Model**: What IS a roofing panel configuration? (structure, attributes, relationships)
    - How does it map to existing configurator data model?
    - What's NorSeam-specific vs reusable from wall panels?
2. **No Validation Requirements Catalog**:
    - What validation rules are needed for architectural panels? For roofing panels?
    - What types of errors currently occur in production?
    - Complexity assessment: Simple range checks vs complex interdependencies?
    - Priority classification: Critical blockers vs warnings vs recommendations?
3. **No Validation Architecture Strategy**:
    - Current NAI system has no validation capability
    - Approach undefined: Build straightforward validation vs adapt existing complex system
    - Implementation complexity not assessed
4. **Integration Delta Undefined**: Existing integrations work for wall panels, but:
    - Which new attributes needed for roofing submissions to Epicor?
    - Do planning systems need new fields for NorSeam workflows?
    - Backward compatibility strategy for mixed panel-type orders?
5. **No User Workflows**: Current vs desired user journey not documented
    - How does roofing configuration differ from wall panel flow?
    - Which UI components are reusable?
6. **No Error Taxonomy**: Types of invalid configurations causing production issues not enumerated
7. **No Foundational Governance Documents**:
    - No Project Brief (orientation, critical elements, success criteria)
    - No Technical Analysis (solution recommendations)
    - No Technical Specification

**Impact:** Cannot estimate extension effort without understanding:

- How much of existing system is reusable
- Roofing-specific complexity vs generic configurator complexity

---

## 🏗 Architectural Decisions

**Core Questions (Constrained by Existing System):**

- **Validation architecture approach**: NAI currently has no validation capability
    - Build straightforward hard-coded validation for critical rules?
    - Adapt existing complex validation system from other NorConfig modules?
    - Decision depends on validation requirements complexity assessment
    - Implementation complexity significant regardless of approach chosen

- **Product model extension**: How to add roofing concepts to current data structure?
  - Does existing parametric model support front/back, laps, notches?
  - Breaking changes vs backward-compatible extension?
- **Knowledge Graph**: Defer unless existing configurator already uses similar patterns
- **Integration patterns**: Must follow existing synchronous/async patterns for consistency
- **Low-code/no-code**: Not viable for initial release
  - Validation rules will be hard-coded (at least for V2.0)
  - Business configurability deferred to future phases

**Assessment:** Multiple architectural decisions remain unresolved:

- Validation strategy selection blocks implementation planning
- Architecture extensibility assessment needed before estimating effort
- Key question: **Can existing architecture handle roofing panels AND validation without major rework?**
  - If YES → timeline challenging but possible
  - If NO → major refactoring required, timeline at high risk

---

## 🔗 Integration Assessment

**Existing Production Integrations (6 systems):**

1. ~~NorConfig (V1.x - currently handling wall panels)~~ → **This IS NorConfig** being extended
2. Epicor (ERP - "100% of submissions") → **Already integrated and working**
3. Planning systems → **Already integrated and working**
4. planMES → **Already integrated and working**
5. Design/CAD tools → **Already integrated and working**
6. ShopFloor/data warehouse → **Already integrated and working**

**Integration Changes Required (Minimal Scope):**

- Add roofing-specific attributes to existing data payloads:
  - Front/back panel indicators
  - Lap configuration values
  - Notch specifications
  - Packaging method codes
- Verify existing validation pipelines handle new attribute ranges
- Ensure downstream systems (Epicor, MES) can consume new fields without breaking

**What's Already Done (No Work Required):**

- ☑ API contracts established
- ☑ Authentication/authorization
- ☑ Error handling/retry logic
- ☑ Data ownership boundaries
- ☑ Integration monitoring/logging

**Assessment:** Integration work estimated at **<10% of timeline** (minor schema extensions).

**Risk:** Existing integrations might reject new roofing-specific values if not backward-compatible. **Action:** Test integration payloads early.

---

## 💡 Priority Contradiction

**Document States Objectives (As Written):**

1. "Deliver configurator producing valid specifications"
2. "Ensure 100% Epicor integration"
3. "Prevent invalid configurations"

**Contradiction Analysis:** These objectives treat **validation perfection** as equal priority to **basic functionality**, which creates unrealistic expectations:

- **Objective 1 + 3 combined** = "Only allow perfectly valid configurations" → Implies 100% rule coverage from day 1
- **Reality**: Current system allows SOME invalid configs → Need to identify WHICH errors are blockers vs nice-to-catch
- **Objective 2** already satisfied (Epicor integration exists in production)

**Why This is Problematic:**

- **Users cannot configure ANY roofing panels** until validation is "perfect" → Feature launch blocked
- **"Valid specification"** is ambiguous: Valid per business rules? Per production capabilities? Per regulatory codes?
- **"Prevent invalid configurations"** is binary language for a spectrum problem:
  - Critical errors (e.g., physically impossible panel dimensions) → Must prevent
  - Production inefficiencies (e.g., non-standard packaging) → Warn but allow
  - Aesthetic sub-optimizations → Document but don't block

**Pragmatic Reframing:**

1. **P0 (Launch Blocker):** Enable roofing panel configuration in UI
   - Support NorSeam attributes: front/back, laps, notches, packaging methods
   - Generate specifications (even if validation is incomplete)
   - Submit to Epicor using existing integration (add new fields)
2. **P1 (High Business Value):** Prevent critical production errors
   - Catalog current error frequency: Which configs fail in production?
   - Implement top 5-10 validation rules by impact (failure cost × frequency)
   - Target: Reduce production rework by 80%, not eliminate 100% of errors
3. **P2 (Future Enhancement):** Advanced validation & configurability
   - Comprehensive rule coverage (diminishing returns past 80%)
   - Business-editable validation rules (requires tooling investment)
   - Predictive validation (ML-based error detection)

**Trade-off Clarity:** Perfect validation delays roofing panel launch → Lost sales opportunities Vs. Incremental validation allows launch → Gradual error reduction while capturing market

---

## 📋 Proper Project Workflow

**Missing Foundation:** Project currently lacks foundational governance documents and structured workflow. A typical project lifecycle would include:

**Phase 1: Discovery & Clarification (Week 1)**

**Purpose:** Gather information needed to write meaningful Project Brief

**Activities:**

- Stakeholder interviews with Vincent Bérubé, production team, estimators, planning
  - Clarify business priorities and needs
  - Define acceptable trade-offs (speed vs features, scope vs timeline)
  - Identify constraints (resources, dependencies, deadlines)
- Production error cataloging with manufacturing team (Sébastien, Wael)
  - Document types of invalid configurations that occur
  - Classify by severity: Critical blockers vs production inefficiencies vs aesthetic issues
  - Quantify frequency and cost impact
- Validation requirements complexity assessment
  - Analyze error patterns: Simple rule violations vs complex interdependencies?
  - Determine validation sophistication needed
  - Inform validation architecture decision
- Integration smoke testing
  - Prototype roofing panel data structure
  - Test integration with Epicor/MES in dev environment
  - Identify breaking changes or compatibility issues early
- Current architecture review
  - Review NorConfig V1.x codebase and data model
  - Assess extensibility: Can it handle roofing panels?
  - Identify hard-coded assumptions requiring changes

**Deliverables:** Discovery findings document

---

**Phase 2: Project Brief Development & Approval (Week 1-2)**

**Purpose:** Establish project foundation with stakeholder consensus

**Brief Content:**

- **Orientation**: Strategic direction and approach philosophy
  - Example: "Extend existing system incrementally; prioritize speed to market over sophistication"
  - Example: "Backward compatibility non-negotiable; existing workflows must remain unaffected"
- **Critical vs Important vs Deferred Elements**:
  - Critical: Roofing panel configuration, Epicor integration, top production error prevention

    - Important: Broader validation coverage, improved UX
    - Deferred: Business-configurable rules, advanced features
- **Success Criteria** (Development Process):
    - Process milestones: Brief approved by [date], Spec approved by [date], Beta release by [date]
    - Quality gates: Integration tests pass, backward compatibility verified, code review completed
- **Governance Structure**:
    - Close direction: Day-to-day project oversight and tactical decisions
    - Higher direction: Strategic decisions, scope changes, timeline adjustments, escalations
    - Decision thresholds: When to escalate vs resolve locally
- **Constraints**: Timeline, resources, technical, business

**Deliverable:** Approved Project Brief (signed by Vincent Bérubé and key stakeholders)

---

**Phase 3: Technical Analysis (Week 2-3)**

**Purpose:** Develop solution recommendations based on Brief orientation

**Analysis Content:**

- **Validation Strategy Recommendation**:
    - Based on complexity assessment from Discovery phase
    - Option A: Build straightforward hard-coded validation (if rules are simple)
    - Option B: Adapt existing complex validation system (if complex interdependencies)
    - Recommendation with rationale, effort estimate, risk assessment
- **Architecture Approach**:
    - Product model extension strategy (backward-compatible vs breaking changes)
    - Integration design (schema changes, payload updates)
    - UI extensions needed (roofing-specific inputs within existing interface patterns)
- **Implementation Phasing**:
    - Development sequence aligned with Brief priorities
    - Risk mitigation strategies
    - Resource allocation plan
- **Risk Assessment**:
    - Technical risks, timeline risks, integration risks
    - Mitigation strategies for each

**Deliverable:** Technical Analysis document with solution recommendations

---

**Phase 4: Technical Specification (Week 3-4)**

**Purpose:** Detailed design based on approved Analysis recommendations

**Specification Content:**

- NorSeam product domain model (attributes, relationships, constraints)
- Validation rules specification (detailed logic, priority, error messages)
- Architecture design (data model, API contracts, component structure)

- Integration specifications (Epicor payload schema, MES updates, backward compatibility approach)
- UI extensions specification (roofing-specific input fields, form layouts, validation feedback using existing UI patterns)
- Test strategy (unit, integration, regression, acceptance criteria)

**Deliverable:** Approved Technical Specification

---

**Phase 5: Development & Deployment (Week 5-16)**

**Dependent on:** Approved Specification and realistic scope

**Development Activities:**

- Implementation per Specification
- Iterative testing and validation
- Integration verification
- Beta testing with internal users (Nancy, Norman)
- Production pilot with limited scope
- Full rollout

---

## ⏱ Timeline Impact Analysis

**Proper workflow time allocation:**

- Phase 1 (Discovery): 1 week
- Phase 2 (Brief): 1 week (including approval)
- Phase 3 (Analysis): 1-2 weeks (including approval)
- Phase 4 (Specification): 1-2 weeks (including approval)
- **Total Planning: 4-6 weeks**

**Development time available:**

- Original runway: 16 weeks (April 6 to August 2026)
- Planning phases: 4-6 weeks
- **Coding/testing/deployment: 10-12 weeks**

**Comparison:** Establishing proper project governance consumes 25-35% of available timeline. However, skipping these phases increases risk of:

- Building wrong solution (misaligned with business needs)
- Underestimating complexity (validation architecture decision unmade)
- Scope creep (no agreed boundaries)
- Failed delivery (unrealistic expectations)

---

## ⚠ Red Flags

1. **Status "Idea to be Scoped"** at T-minus 4 months without foundational governance
    - No Project Brief defining orientation, priorities, success criteria

    - No Technical Analysis evaluating solution approaches
    - No Technical Specification providing implementation blueprint
2. **Validation architecture decision unmade**
    - NAI currently has no validation capability
    - Validation strategy undefined (build simple vs adapt complex system)
    - Implementation complexity not assessed
    - Blocks accurate effort estimation and timeline planning
3. **No technical architect assigned** despite extending production-critical system
4. **"Possible consulting support needed"** mentioned but not actioned
5. **No risks documented** despite:
    - Extending live production system with tight timeline
    - Validation development from scratch
    - Architectural decisions still pending
    - Backward compatibility requirements
6. **Comments suggest splitting into phases** but no phase definitions exist
7. **Resource allocation undefined** ("Resources: TBD") with 16-week runway
8. **Governance structure undefined**
    - No defined close direction (day-to-day oversight)
    - No defined higher direction (strategic decisions, escalations)
    - No decision-making authority established

---

## 🎯 Bottom Line

**Project feasibility uncertain due to missing foundational work.** Current state analysis:

**Critical Gaps:**

1. **No Foundational Governance Documents**:
    - Missing Project Brief: Orientation, priorities, and success criteria undefined
    - Missing Technical Analysis: Solution approach undecided (especially validation strategy)
    - Missing Technical Specification: Implementation details not designed
    - **Impact**: Cannot accurately estimate effort, timeline, or resources
2. **Validation Architecture Decision Blocking**:
    - NAI currently has no validation capability (must be built)
    - Validation complexity not assessed (simple rules vs complex interdependencies)
    - Implementation approach undefined (build straightforward vs adapt complex system)
    - **Impact**: Major architectural decision unmade; significant effort estimation uncertainty
3. **Timeline Accounting Issue**:
    - Proper project workflow requires 4-6 weeks (Discovery → Brief → Analysis → Spec)
    - Available development time: Only 10-12 weeks from 16-week runway
    - Validation development + roofing panel functionality in 10-12 weeks = high risk
    - **Impact**: Timeline appears optimistic without scope reduction or extension

**What Proper Workflow Would Include:**

**Phase 1: Discovery (Week 1)**

- Stakeholder clarification of priorities, needs, constraints
- Production error cataloging and validation complexity assessment
- Integration smoke testing and architecture review
- **Deliverable**: Discovery findings

**Phase 2: Project Brief (Week 1-2)**

- Document orientation, critical/important/deferred elements
- Define process success criteria and governance structure
- Establish decision-making authority and escalation paths
- **Deliverable**: Approved Brief with stakeholder sign-off

**Phase 3: Technical Analysis (Week 2-3)**

- Validation strategy recommendation based on complexity assessment
- Architecture approach and integration design
- Risk assessment and mitigation strategies
- **Deliverable**: Technical Analysis with solution recommendations

**Phase 4: Technical Specification (Week 3-4)**

- Detailed design of domain model, validation rules, architecture, integrations, UI/UX
- Test strategy and acceptance criteria
- **Deliverable**: Approved Specification

**Phase 5: Development (Week 5-16)**

- Implementation, testing, deployment per Specification
- Actual development time: 10-12 weeks

**Risk Assessment:**

**Timeline Viability:**

- Proper governance workflow leaves 10-12 weeks for development
- Scope includes: Roofing panels + Validation architecture + Integration updates + UI enhancements
- **Assessment**: Timeline at high risk without significant scope reduction or timeline extension

**Architectural Risk:**

- Validation strategy decision unmade (blocks accurate estimation)
- Architecture extensibility unknown (blocks design decisions)
- Implementation complexity significant regardless of approach
- **Assessment**: Technical risk high until architectural decisions resolved

**Governance Risk:**

- No defined oversight structure (close vs higher direction)

- No decision-making authority established
- No escalation paths defined
- **Assessment**: Process risk high; increases likelihood of misalignment and rework

**Recommendation:**

Establish proper project foundation before committing to August 2026 delivery:

1. Execute Discovery phase to assess validation complexity and architecture compatibility
2. Develop Project Brief with realistic scope based on findings
3. Validate timeline feasibility after Brief and Analysis are complete
4. Consider timeline extension or scope reduction based on Technical Analysis
5. Establish governance structure with clear decision authority and escalation paths